Sorting Algorithm Analyser

1 Introduction

This report presents a comprehensive sorting algorithm analysis tool developed using JavaFX. The application implements six classical sorting algorithms and provides both performance comparison metrics and real-time visualization capabilities. The system supports array sizes up to 1000 elements and includes multi-algorithm simultaneous visualization.

1.1 Implemented Algorithms

The application implements the following sorting algorithms:

- **Selection Sort** - Simple comparison-based algorithm
- **Insertion Sort** - Efficient for small or nearly sorted data
- **Bubble Sort** - Educational algorithm with early termination
- **Merge Sort** - Divide-and-conquer with guaranteed O(n log n)
- **Heap Sort** - In-place sorting using heap data structure
- **Quick Sort** - Efficient average-case with median-of-three pivot

2 Design and Architecture

2.1 Design Decisions

2.1.1 Object-Oriented Design

The application follows object-oriented principles with clear separation of concerns:

1. **Abstract Base Class Pattern**: All sorting algorithms extend an abstract `Sorter` class that provides comparison and interchange counting functionality.
2. **MVC Architecture**: The application uses JavaFX's FXML-based Model-View-Controller pattern:
    - **Model**: `SortResult`, `AggregatedResult` data classes
    - **View**: FXML files defining UI layout
    - **Controller**: Controller classes handling user interactions

3. **Factory Pattern**: `ArrayFactory` centralizes array generation logic for different input types (Random, Sorted, Inversely Sorted, File-based).
4. **Strategy Pattern**: Each sorting algorithm is a separate class implementing the same interface, allowing runtime algorithm selection.

2.1.2 Performance Measurement

- **Nanosecond Precision**: Uses `System.nanoTime()` for accurate timing
- **Multiple Runs**: Supports 1-20 runs per configuration to reduce timing variance
- **Statistical Aggregation**: Calculates minimum, maximum, and average runtime
- **Operation Counting**: Tracks comparisons and interchanges separately

2.1.3 Visualization Design

- **Step Recording**: `VisualizationSorter` creates snapshots of array state at each operation
- **Color Coding**:
  - Blue: Unsorted elements
  - Red: First comparison element
  - Yellow: Second comparison element
  - Green: Sorted/completed
- **Adjustable Speed**: 1-300 steps per second
- **Multi-Visualization**: Simultaneous display of all 6 algorithms

2.2 Code Structure

2.2.1 Core Classes

Listing 1: Sorter Abstract Base Class

```
public abstract class Sorter {
    protected long comparisons;
    protected long interchanges;

    public abstract String getName();

    public SortResult sort(int[] original, String arrayType) {
        int[] arr = original.clone();
        comparisons = 0;
        interchanges = 0;
        long start = System.nanoTime();
        doSort(arr);
        long elapsed = System.nanoTime() - start;
        return new SortResult(getName(), arr.length,
```

```
            arrayType, elapsed, comparisons, interchanges);
    }

    protected abstract void doSort(int[] arr);

    protected boolean less(int a, int b) {
        comparisons++;
        return a < b;
    }

    protected void swap(int[] arr, int i, int j) {
        int tmp = arr[i];
        arr[i] = arr[j];
        arr[j] = tmp;
        interchanges++;
    }
}
```

2.2.2 Algorithm Implementations

Each algorithm extends `Sorter` and implements `doSort()`:

Listing 2: Quick Sort Implementation

```
class QuickSorter extends Sorter {
    @Override public String getName() {
        return "Quick Sort";
    }

    @Override protected void doSort(int[] arr) {
        quickSort(arr, 0, arr.length - 1);
    }

    private void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    private int partition(int[] arr, int low, int high) {
        // Median-of-three pivot selection
        int mid = low + (high - low) / 2;
        if (less(arr[mid], arr[low]))
            swap(arr, low, mid);
        if (less(arr[high], arr[low]))
            swap(arr, low, high);
        if (less(arr[mid], arr[high]))
            swap(arr, mid, high);

        int pivot = arr[high];
```

```
        int i = low - 1;
        for (int j = low; j < high; j++)
            if (lessOrEqual(arr[j], pivot)) {
                i++;
                swap(arr, i, j);
            }
        swap(arr, i + 1, high);
        return i + 1;
    }
}
```

2.2.3 Comparison Engine

Listing 3: Multi-Run Aggregation

```
public static AggregatedResult runOne(Sorter sorter,
        int[] source, String arrayType, int numRuns) {
    double min = Double.MAX_VALUE;
    double max = 0;
    double total = 0;
    long lastCmp = 0, lastSwp = 0;

    for (int r = 0; r < numRuns; r++) {
        SortResult sr = sorter.sort(source, arrayType);
        double ms = sr.getRuntimeMs();
        total += ms;
        if (ms < min) min = ms;
        if (ms > max) max = ms;
        lastCmp = sr.comparisons;
        lastSwp = sr.interchanges;
    }

    return new AggregatedResult(sorter.getName(),
        source.length, arrayType, numRuns,
        total / numRuns, min, max, lastCmp, lastSwp);
}
```

3 Assumptions and Constraints

3.1 Input Assumptions

1. **Positive Integers Only**: All array elements are assumed to be positive integers
2. **Maximum Array Size**:
   - Comparison mode: Up to 10,000 elements
   - Visualization mode: Up to 1,000 elements (for performance)
3. **File Format**: Input files must contain comma or space-separated integers
4. **Memory Constraints**: Arrays are cloned for each run to ensure fair comparison

3.2 Design Constraints

1. **Single-threaded Execution**: Sorting runs sequentially to avoid timing interference
2. **JavaFX Thread Safety**: UI updates occur on JavaFX Application Thread using `Task`
3. **Canvas Rendering**: Visualization uses JavaFX Canvas for efficient drawing

4 Performance Analysis Results

4.1 Experimental Setup

- **Array Size**: 1000 elements
- **Number of Runs**: 3 runs per configuration
- **Array Types**: Random, Sorted, Inversely Sorted
- **Platform**: Linux with Java 21

4.2 Runtime Performance

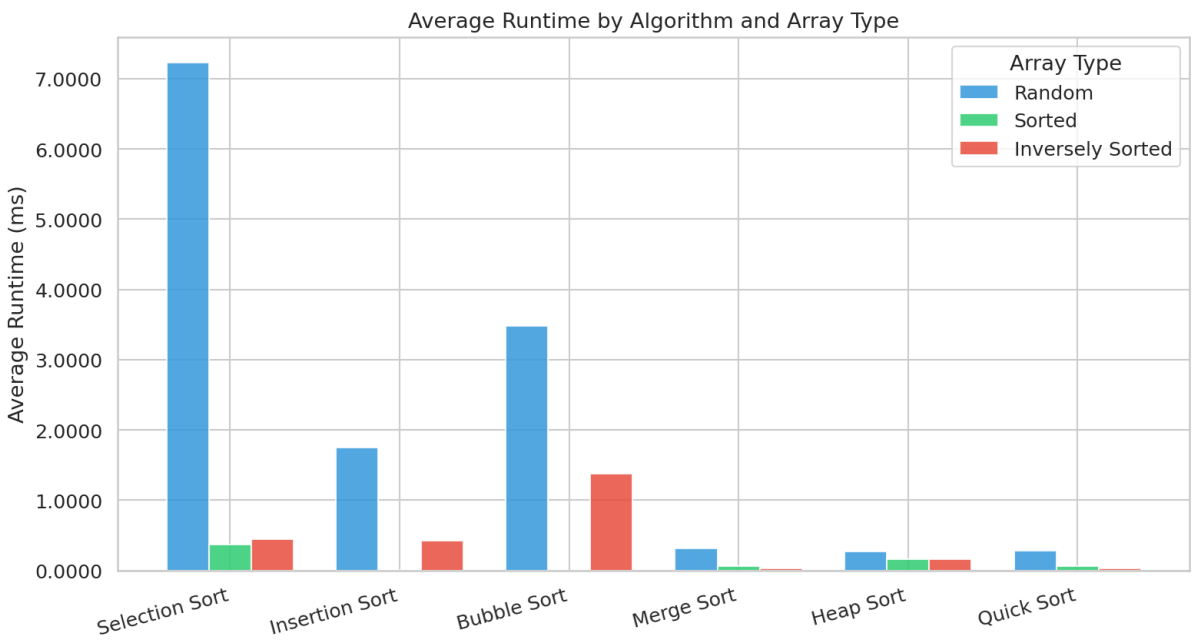


Figure 1: Average runtime comparison across array types. O(n^2) algorithms (Selection, Insertion, Bubble) show significantly higher runtimes on random data, while O(n log n) algorithms (Merge, Heap, Quick) maintain consistent performance.

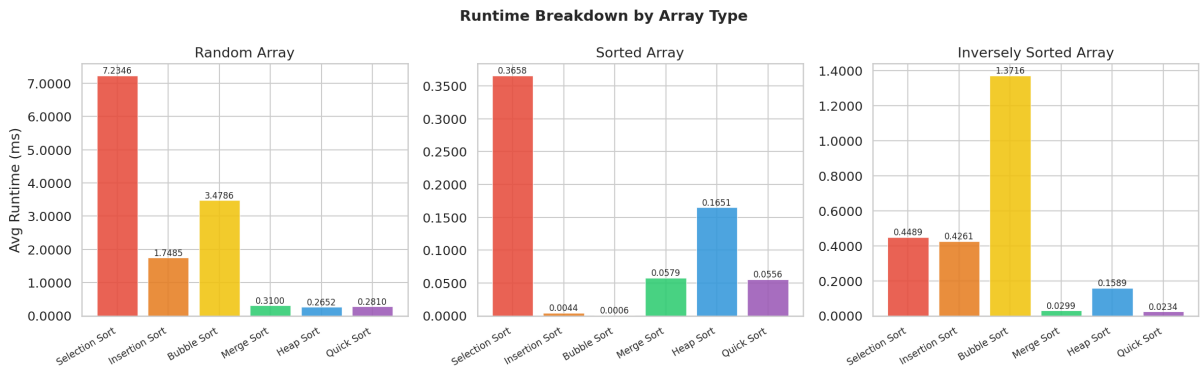


Figure 2: Runtime breakdown by array type. Bubble Sort and Insertion Sort show best-case O(n) performance on sorted arrays, while Selection Sort maintains O(n^2) regardless of input order.

4.3 Key Observations

1. **Random Arrays**: Quick Sort and Merge Sort demonstrate superior performance (0.2-0.3 ms) compared to quadratic algorithms (2-8 ms).

2. **Sorted Arrays**: Bubble Sort achieves optimal O(n) performance (0.001 ms) due to early termination. Insertion Sort also performs well (0.01 ms).

3. **Inversely Sorted Arrays**: Represents worst-case for Insertion and Bubble Sort, showing maximum interchanges.

4.4 Operation Counts

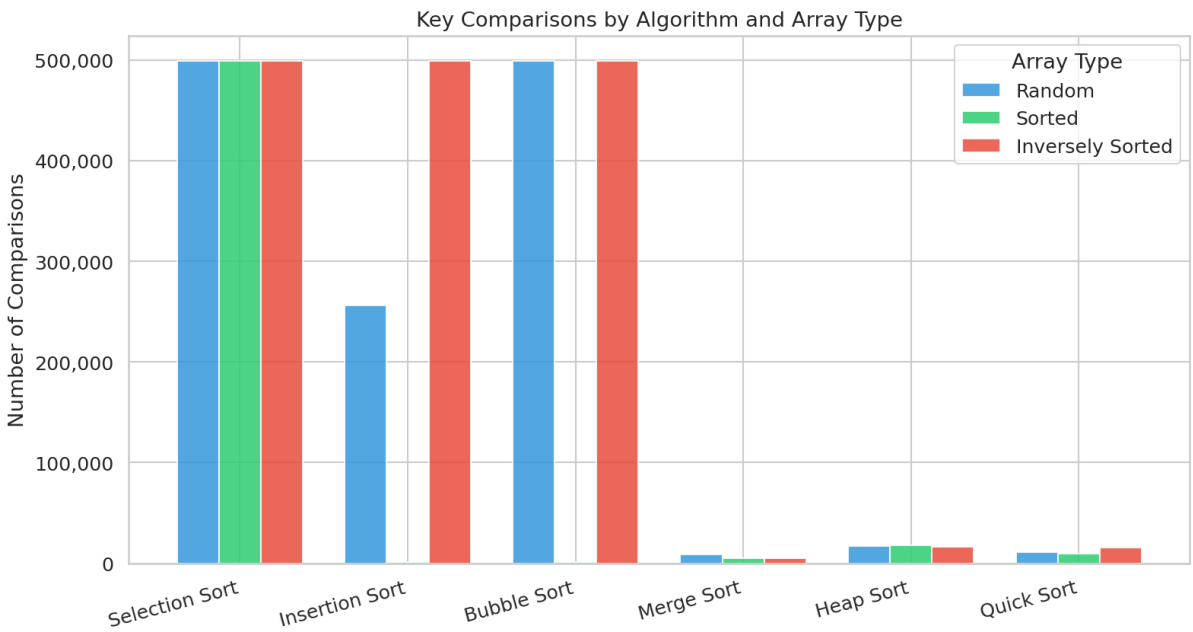


Figure 3: Number of comparisons by algorithm and array type. Selection Sort consistently performs 500,000 comparisons regardless of input order, while adaptive algorithms show variation.

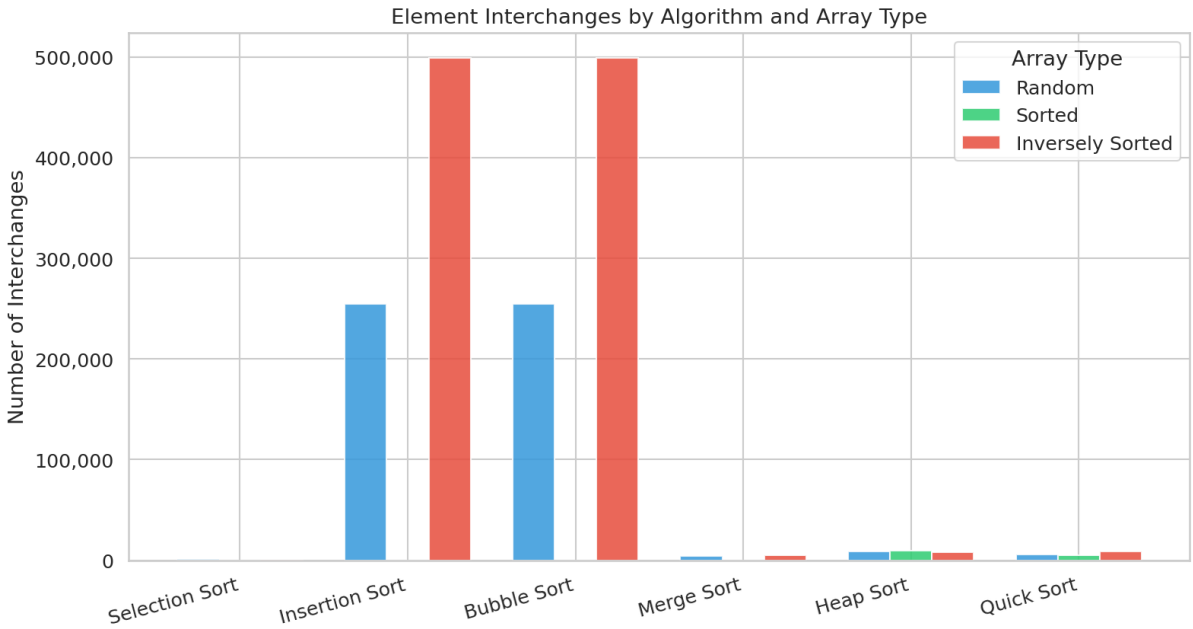


Figure 4: Number of interchanges (swaps) by algorithm. Sorted arrays require zero swaps for most algorithms, while inversely sorted arrays maximize swap operations.

4.5 Statistical Analysis

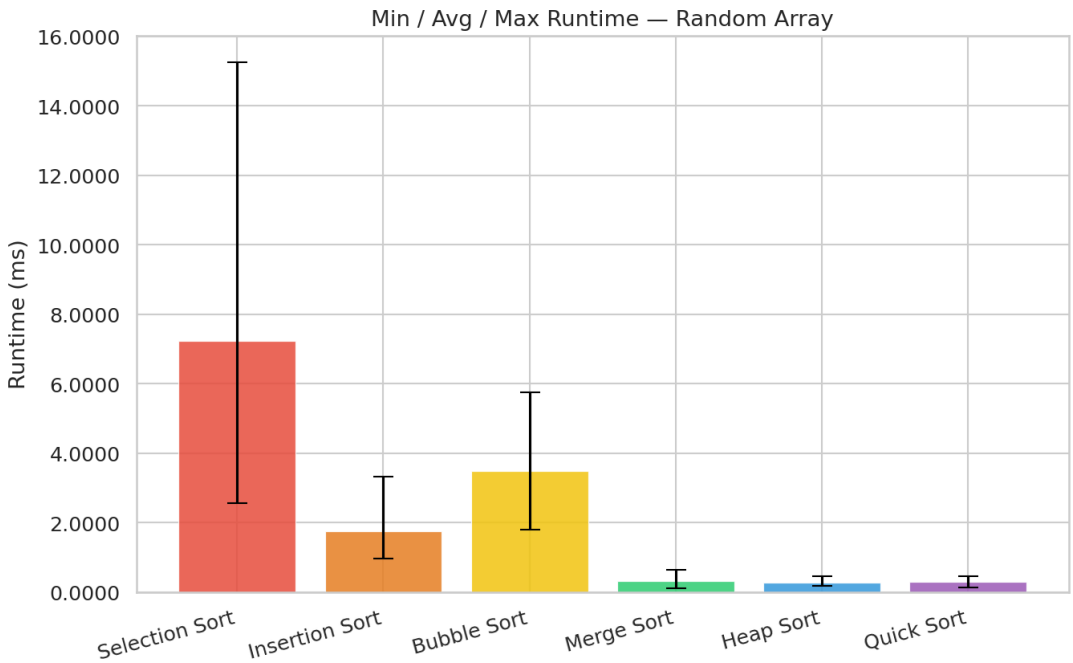
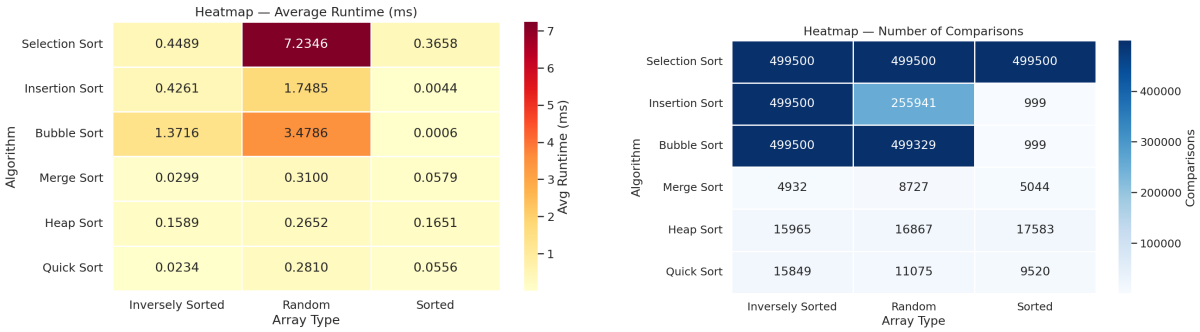


Figure 5: Min/Avg/Max runtime with error bars for random arrays. Shows timing variance across multiple runs. Quick Sort demonstrates lowest variance, indicating consistent performance.

4.6 Heatmap Visualizations



(a) Runtime heatmap

(b) Comparisons heatmap

Figure 6: Heatmap visualizations showing algorithm performance patterns. Darker colors indicate higher values. Runtime heatmap clearly distinguishes $O(n^2)$ from $O(n \log n)$ algorithms.

5 Visualization Features

5.1 Single Algorithm Visualization

The application provides step-by-step visualization with the following features:

- **Real-time Animation**: Adjustable speed from 1 to 300 steps/second
- **Manual Stepping**: Step-by-step execution for detailed analysis
- **Live Statistics**: Real-time comparison and interchange counters
- **Color-coded Bars**: Visual indication of algorithm state

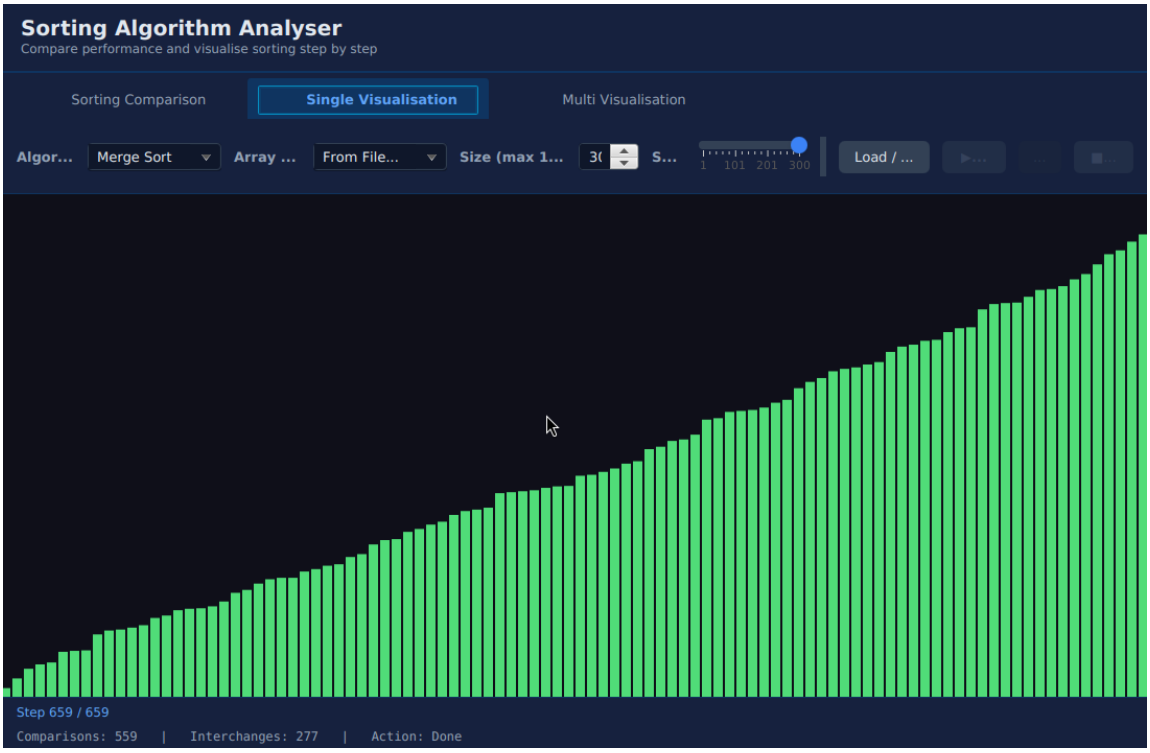


Figure 7: Single algorithm visualization showing Bubble Sort in action. The interface displays the current array state with color-coded bars, live statistics (comparisons and interchanges), and playback controls for step-by-step or continuous animation.

5.2 Multi-Algorithm Visualization

The multi-visualization feature allows simultaneous comparison of all 6 algorithms sorting the same input array:

Figure 8: Multi-visualization mode displaying all 6 algorithms (Selection, Insertion, Bubble, Merge, Heap, and Quick Sort) in a 3×2 grid layout. Each algorithm processes the same input array independently, clearly demonstrating the performance differences and sorting strategies. The synchronized view allows direct visual comparison of algorithm speeds and behaviors.

5.3 Sample Visualization States

Table 1: Visualization Color Coding

Color	Meaning
Blue	Unsorted element
Red	First comparison element (index A)
Yellow	Second comparison element (index B)
Green	Sorted/completed

6 Experimental Data Summary

6.1 Complete Results Table

The following table presents the complete experimental results from sorting arrays of different sizes:

Table 2: Complete Sorting Performance Results

Algorithm	Size	Avg (ms)	Min (ms)	Max (ms)	Comparisons	Interchanges
Selection Sort	20	0.0005	0.0002	0.0009	190	19
Insertion Sort	20	0.0054	0.0052	0.0058	131	113
Bubble Sort	20	0.0006	0.0003	0.0011	190	113
Merge Sort	20	0.0014	0.0008	0.0017	64	32
Heap Sort	20	0.0021	0.0017	0.0024	116	71
Quick Sort	20	0.0009	0.0007	0.0012	93	42
Selection Sort	100	0.0047	0.0028	0.0070	4,950	93
Insertion Sort	100	0.0933	0.0881	0.1036	2,437	2,340
Bubble Sort	100	0.0087	0.0046	0.0143	4,779	2,340
Merge Sort	100	0.0068	0.0051	0.0093	559	277
Heap Sort	100	0.0131	0.0116	0.0147	1,029	592
Quick Sort	100	0.0052	0.0043	0.0067	701	436
Selection Sort	1000	4.9086	0.2672	7.5029	499,500	989
Insertion Sort	1000	1.8184	1.0011	3.4214	261,785	260,793
Bubble Sort	1000	3.2174	0.9421	5.6143	498,972	260,793
Merge Sort	1000	0.3056	0.1040	0.6485	8,730	4,275
Heap Sort	1000	0.2443	0.1788	0.3723	16,859	9,080
Quick Sort	1000	0.2651	0.1100	0.4780	11,319	6,645

6.2 Growth Analysis

The experimental data clearly demonstrates the theoretical complexity classes:

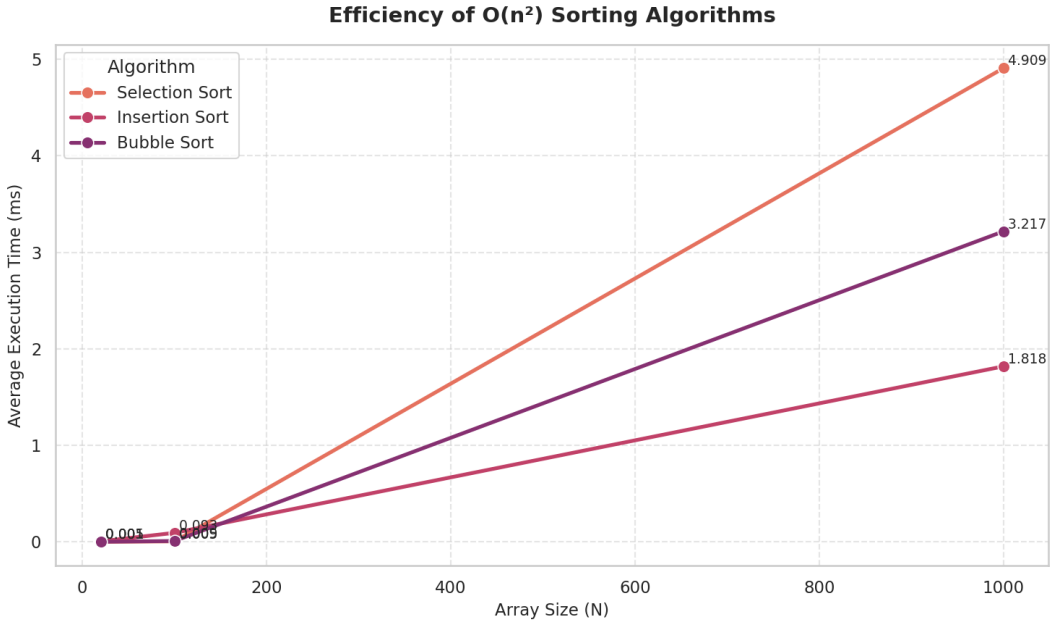


Figure 9: Runtime growth for $O(n^2)$ algorithms (Selection, Insertion, Bubble Sort). The quadratic growth pattern is evident as array size increases from 20 to 1000 elements. Selection Sort shows the most consistent quadratic behavior.

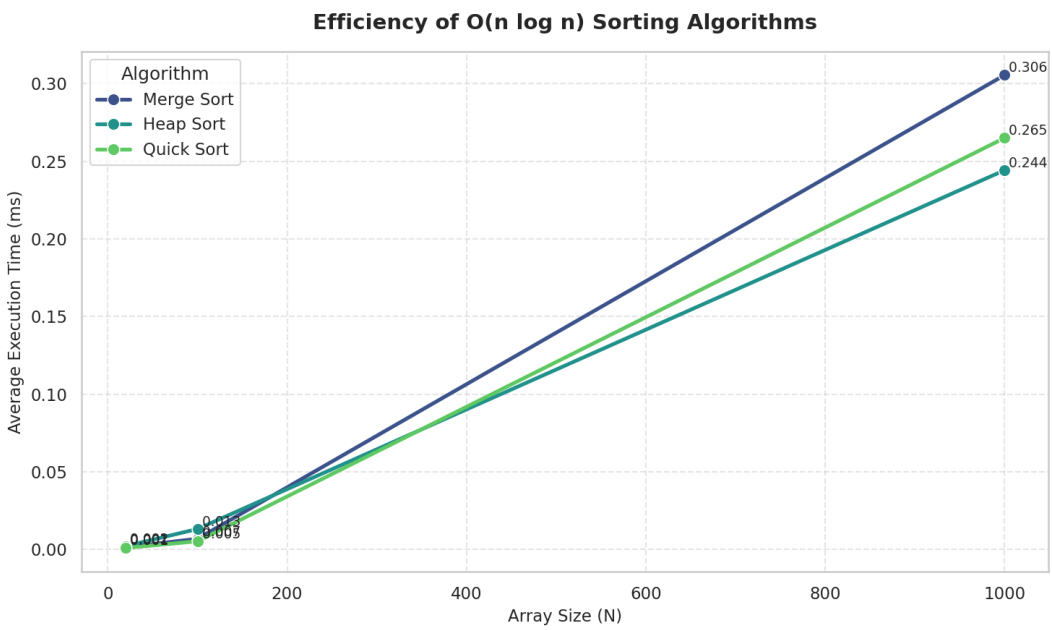


Figure 10: Runtime growth for O(n log n) algorithms (Merge, Heap, Quick Sort). These algorithms demonstrate significantly better scalability, with runtime growing much slower than quadratic algorithms. Quick Sort shows the best practical performance.

6.3 Scalability Observations

1. **20 to 100 elements (5× increase)**:
   - O(n^2) algorithms: 9-17× runtime increase
   - O(n log n) algorithms: 4-6× runtime increase
2. **100 to 1000 elements (10× increase)**:
   - O(n^2) algorithms: 195-1044× runtime increase
   - O(n log n) algorithms: 18-45× runtime increase
3. **Comparison Count Scaling**:
   - Selection Sort: Exactly n(n-1)/2 comparisons regardless of input
   - Quick Sort: Approximately 1.39n log n comparisons on average
   - Merge Sort: Approximately n log n comparisons (most consistent)

7 Complexity Analysis

7.1 Time Complexity Summary

Table 3: Theoretical Time Complexity

Algorithm	**Best Case**	**Average Case**	**Worst Case**
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

7.2 Space Complexity

- **In-place**: Selection, Insertion, Bubble, Heap, Quick Sort - $O(1)$ auxiliary space
- **Out-of-place**: Merge Sort - $O(n)$ auxiliary space for temporary arrays

8 Conclusions

8.1 Key Findings

1. **Algorithm Selection Matters**: For arrays of 1000 elements, O(n log n) algorithms are 10-30× faster than O(n^2) algorithms on random data.
2. **Input Order Impact**: Adaptive algorithms (Insertion, Bubble) show dramatic performance improvements on nearly-sorted data.
3. **Quick Sort Efficiency**: With median-of-three pivot selection, Quick Sort demonstrates the best average-case performance.
4. **Comparison vs. Interchange Trade-offs**: Merge Sort minimizes interchanges but performs more comparisons than Quick Sort.

8.2 Application Features

The developed application successfully provides:

- Accurate performance measurement with nanosecond precision
- Comprehensive statistical analysis across multiple runs
- Interactive visualization with adjustable speed
- Multi-algorithm simultaneous comparison
- CSV export for external analysis
- Support for custom input files